Token to features

Text features as Represent as follow:

- Integers
- One-hot vectors
- Word embeddings

- Bag of words
- Tf-idf

Token to features

Integers

Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

Token to features

Integers

+ Simple
- Ordering: little semantic sense

hand		happy		zebra
615	< ?!	621	< ?!	1000

One hot encoding

One hot encoding

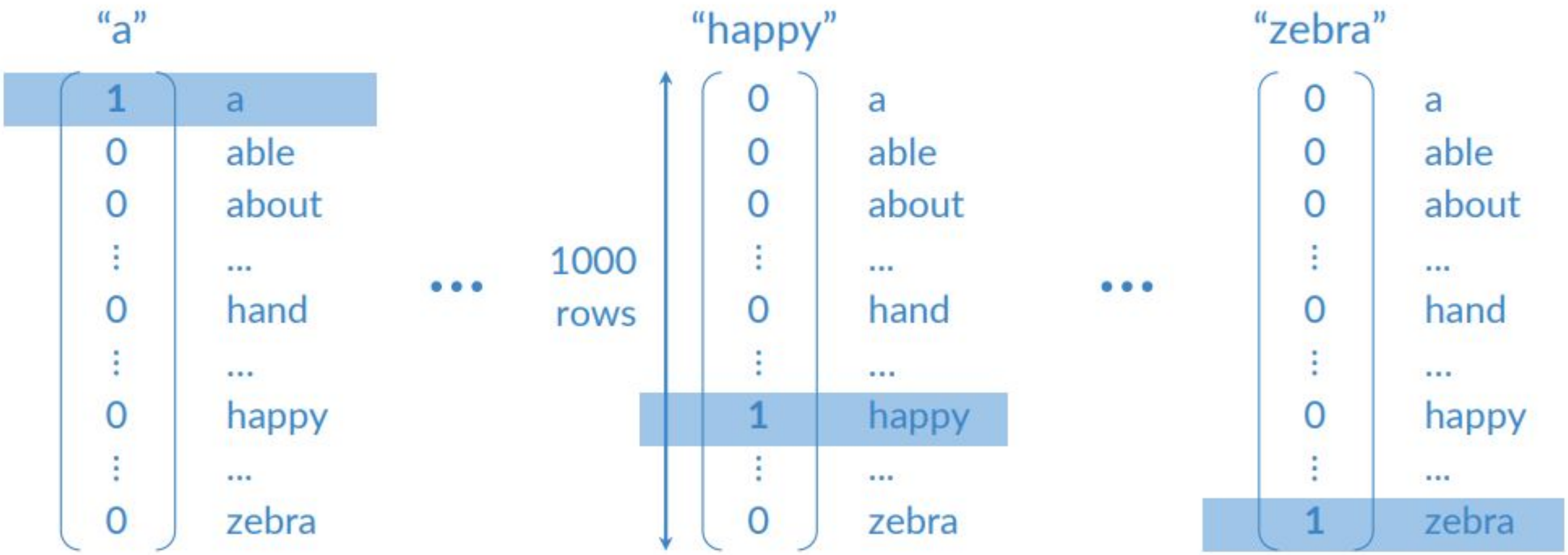
- Converts words to binary vectors.
- Vector length = vocabulary size.
- 1 at the index of the word; 0s elsewhere.
- Unique encoding; no word relationships retained.

Hello, I’m Ironman. I have Friday AI

One Hot Encoding

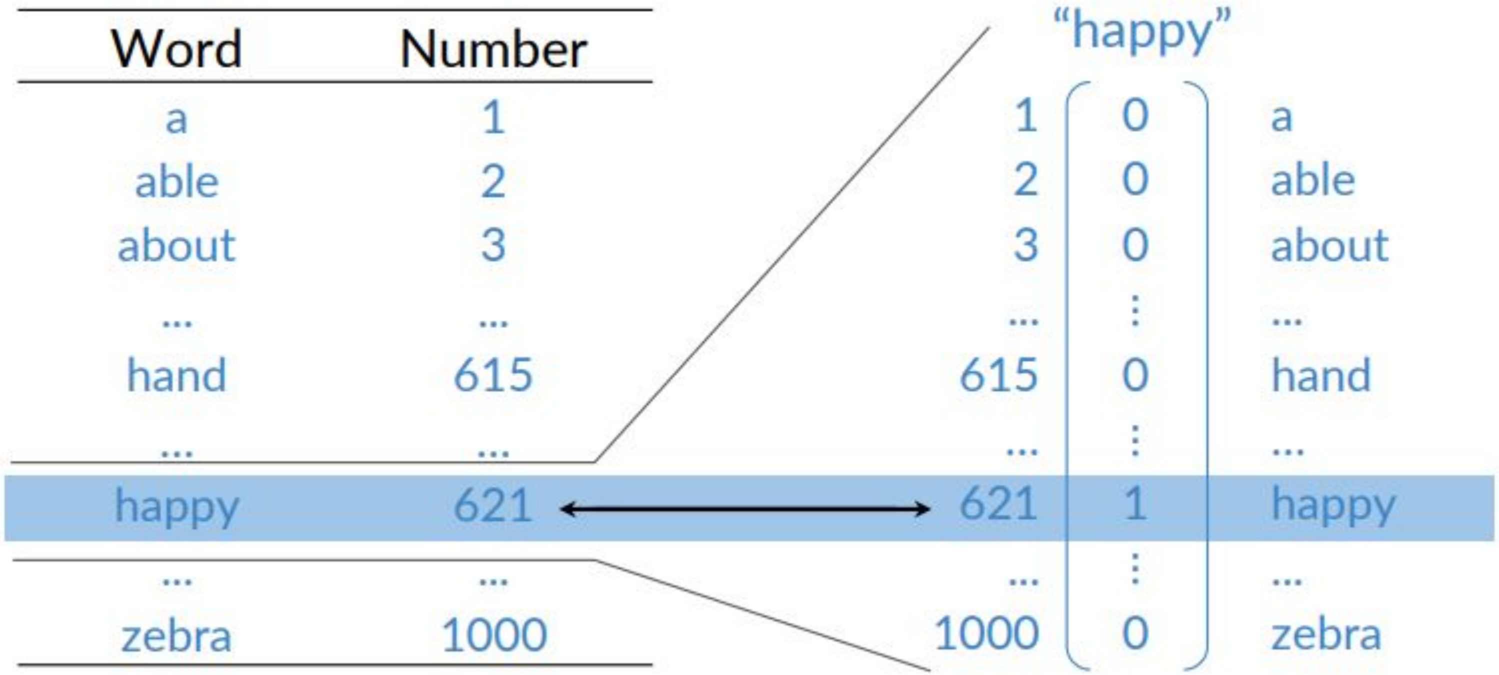
AI	1	0	0	0	0	0	0
Ironman	0	1	0	0	0	0	0
Friday	0	0	1	0	0	0	0
have	0	0	0	1	0	0	0
Hello	0	0	0	0	1	0	0
I	0	0	0	0	0	1	0
I’m	0	0	0	0	0	0	1

One-hot vectors



One - hot vectors

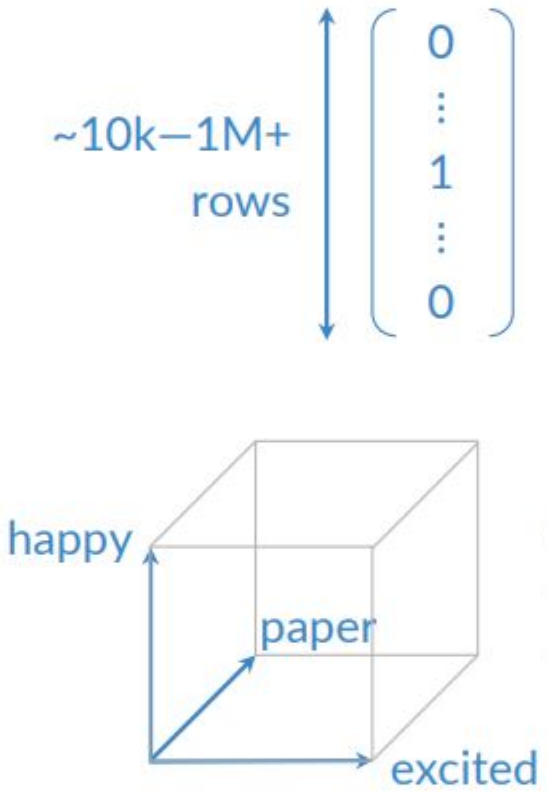
One-hot vectors



One hot vectors

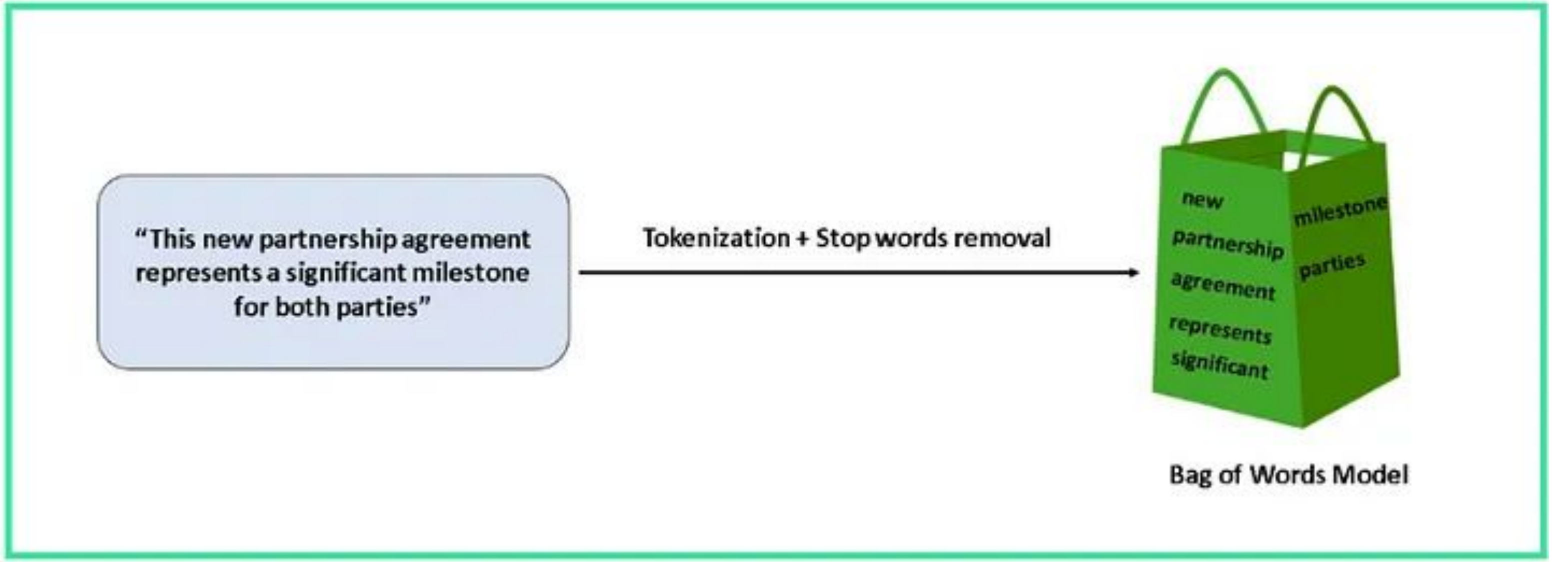
One-hot vectors

+ Simple
+ No implied ordering
- Huge vectors
- No embedded meaning

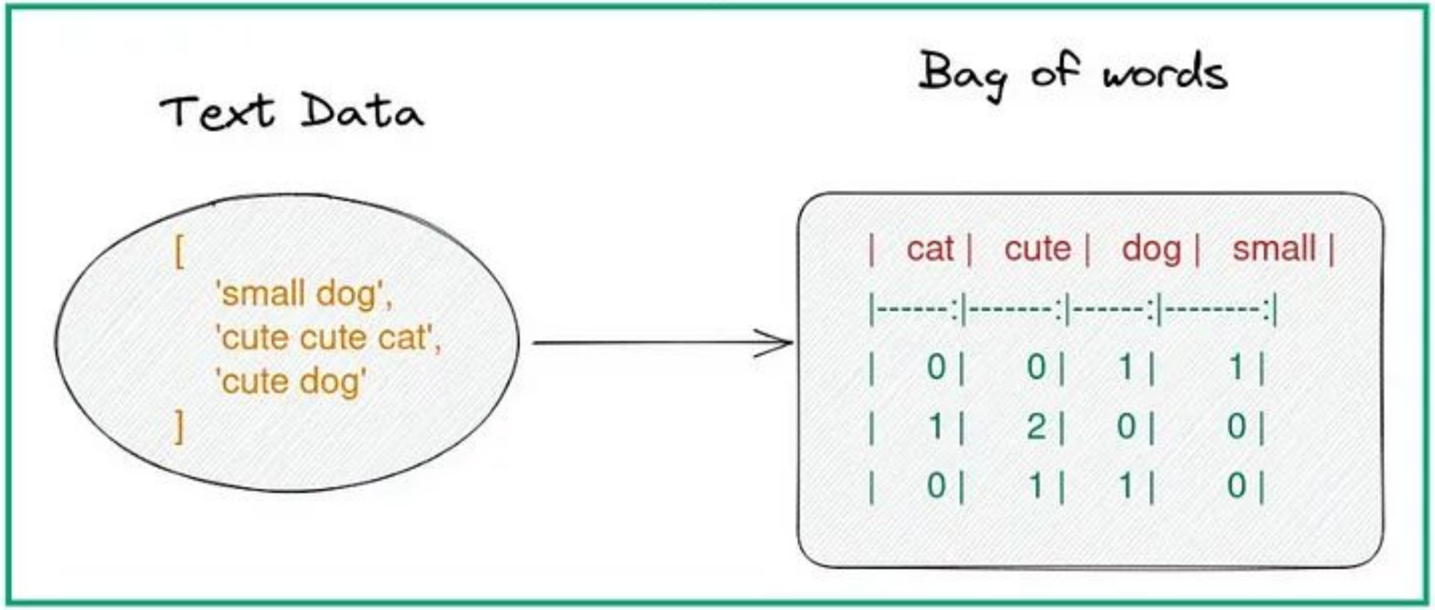


d(paper, excited)
= d(paper, happy)
= d(excited, happy)

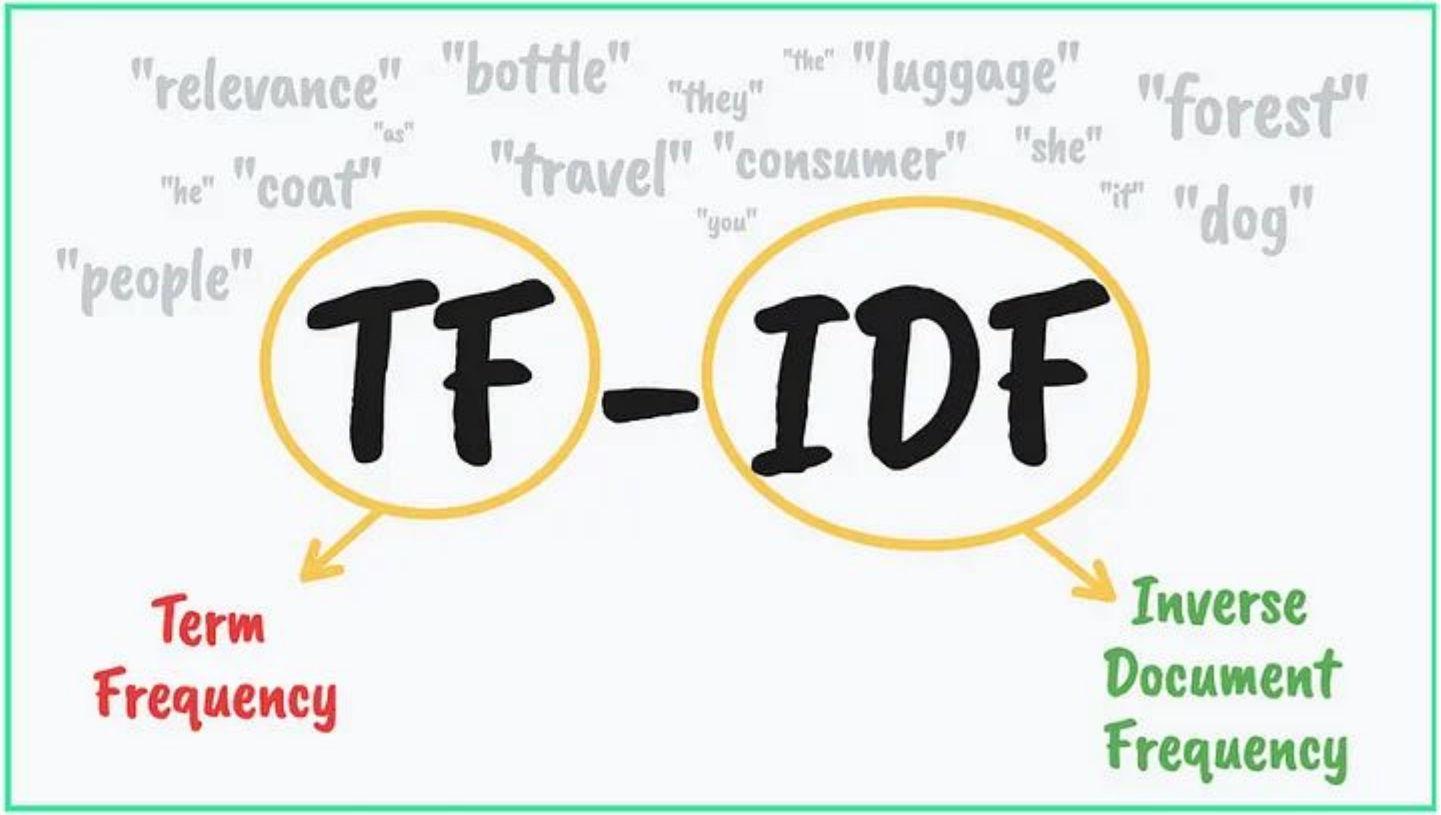
Token to features - Bag of words



Bag of words



Token to features - TF-IDF

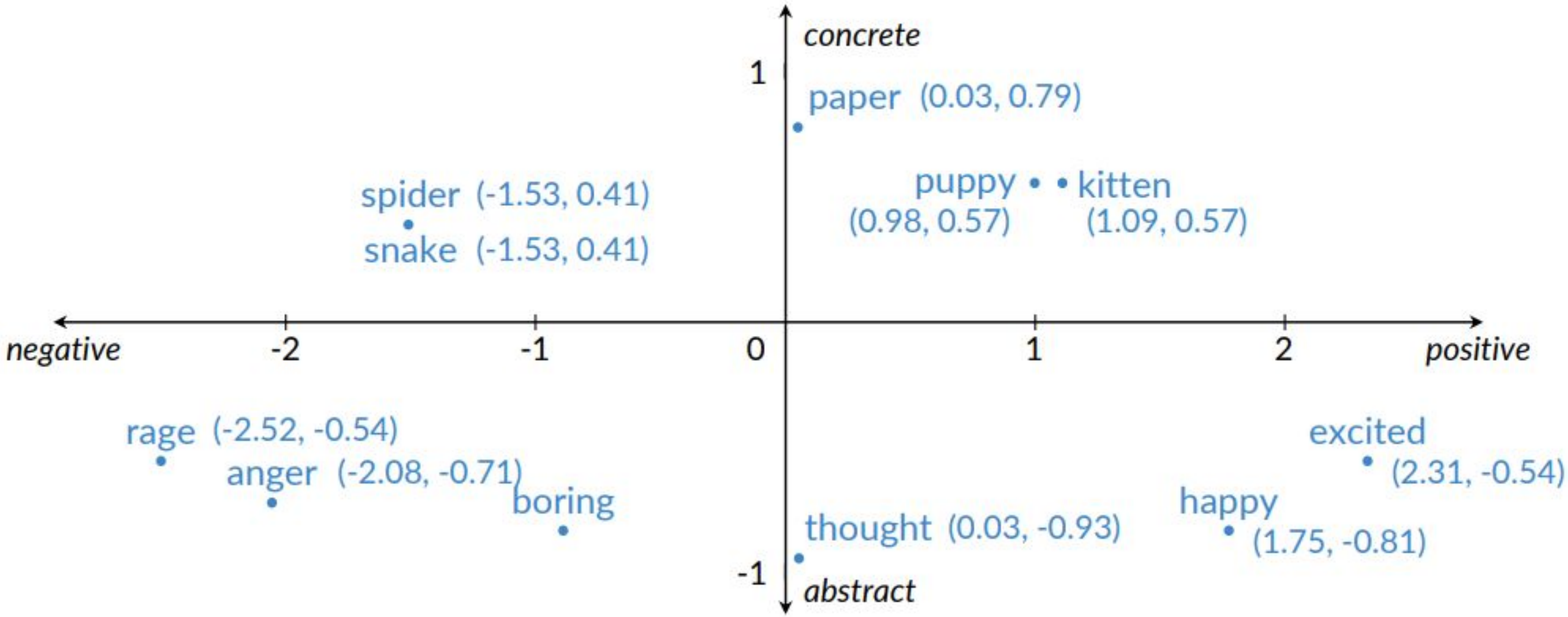


$$TF(t, d) = \frac{\text{number of times t appears in d}}{\text{total number of words in d}}$$

$$IDF(t) = \log \frac{\text{Total number of documents}}{\text{number of documents that contain t}}$$

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Meaning as vectors



Word embedding vectors

+ Low dimension
+ Embed meaning
  - e.g. semantic distance

    forest ≈ tree   forest ≉ ticket

  - e.g. analogies

    Paris:France :: Rome:?

"happy"

~100—~1000 rows

$$\begin{pmatrix} 0.123 \\ \vdots \\ -4.059 \\ \vdots \\ 1.891 \end{pmatrix}$$

Token to features - word vectors

integers

word vectors

one-hot vectors

word embedding vectors

"word vectors"

word embeddings

- Words as integers
- Words as vectors
  - One-hot vectors
  - Word embedding vectors
- Benefits of word embeddings for NLP

Basic word embedding methods

- word2vec (Google, 2013)
  - Continuous bag-of-words (CBOW)
  - Continuous skip-gram / Skip-gram with negative sampling (SGNS)

- Global Vectors (GloVe) (Stanford, 2014)

- fastText (Facebook, 2016)
  - Supports out-of-vocabulary (OOV) words

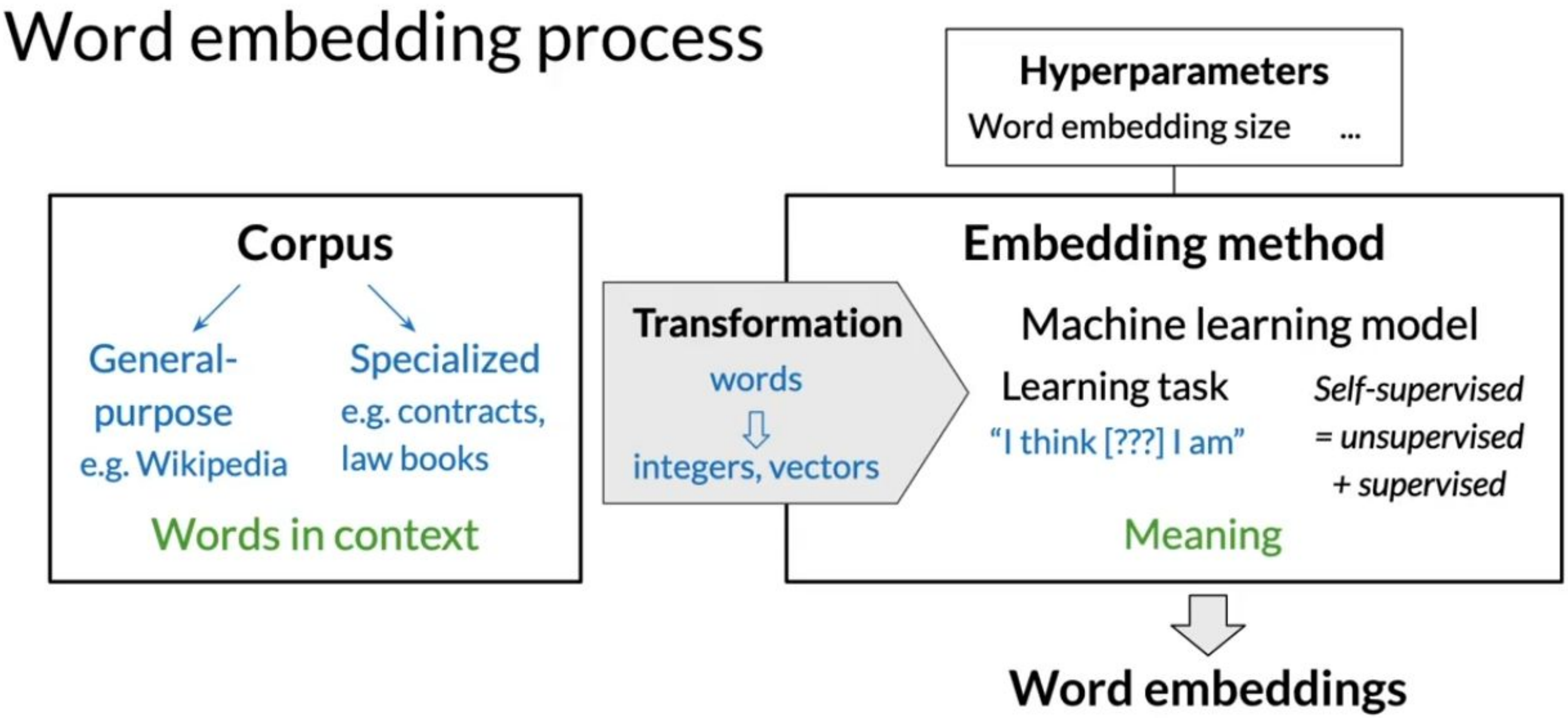
Advanced word embedding methods

Deep learning, contextual embeddings

- BERT (Google, 2018)
- ELMo (Allen Institute for AI, 2018)
- GPT-2 (OpenAI, 2018)

Tunable pre-trained models available

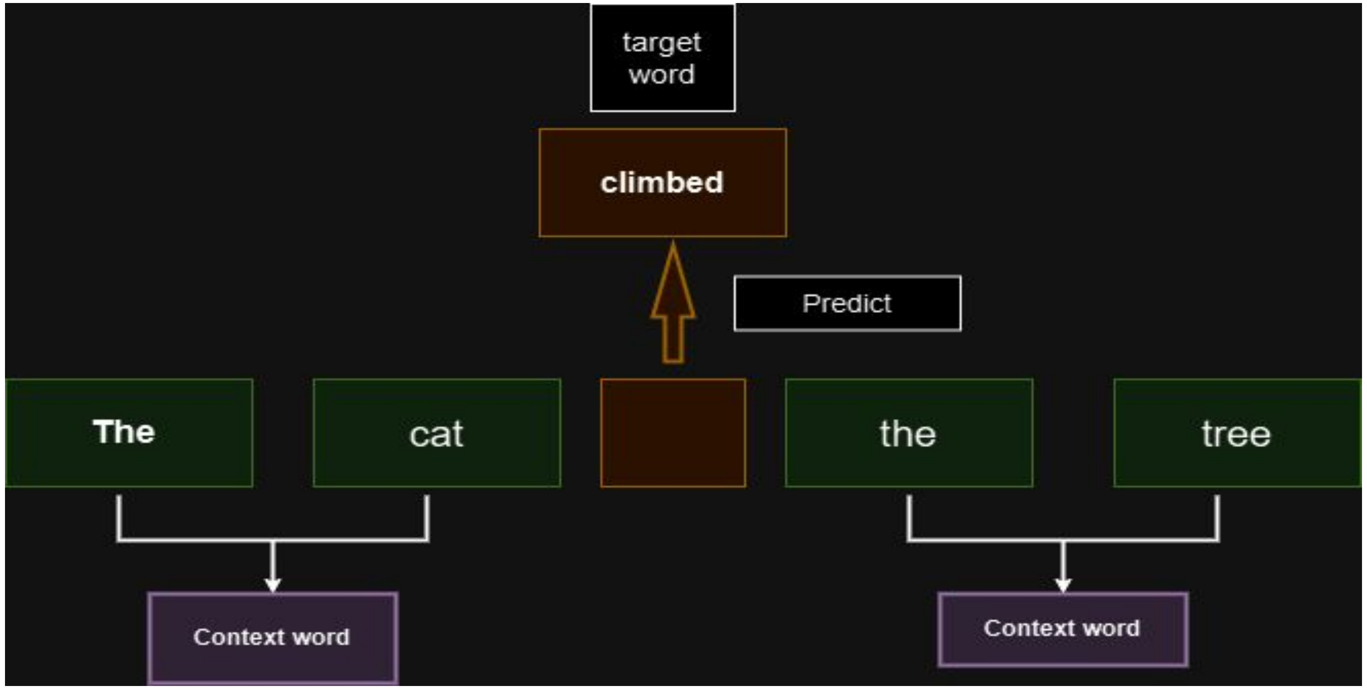
Word Embeddings



Token to features - CBOW

Continuous Bag of Words (CBOW):

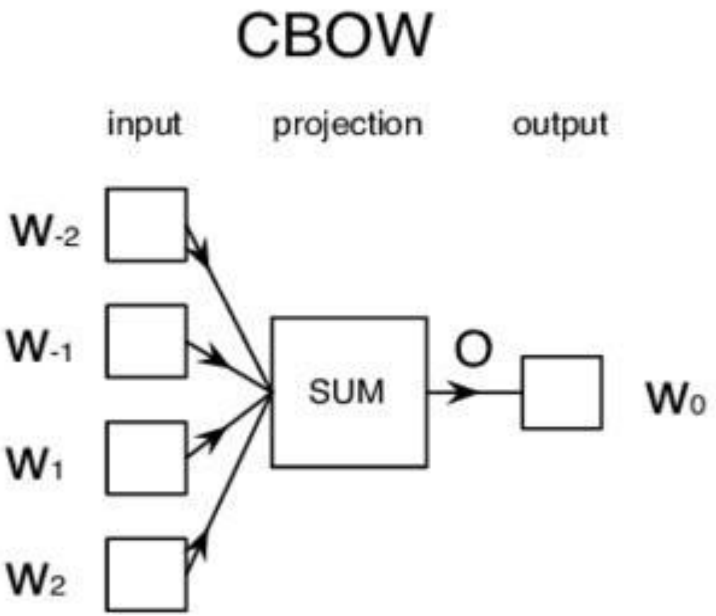
- Predicts a target word using multiple context words.
- Example: Context = "cat," "tree"; Target = "climbed."
- Modification: Input-to-hidden connections replicated CCC times, then averaged in the hidden layer.

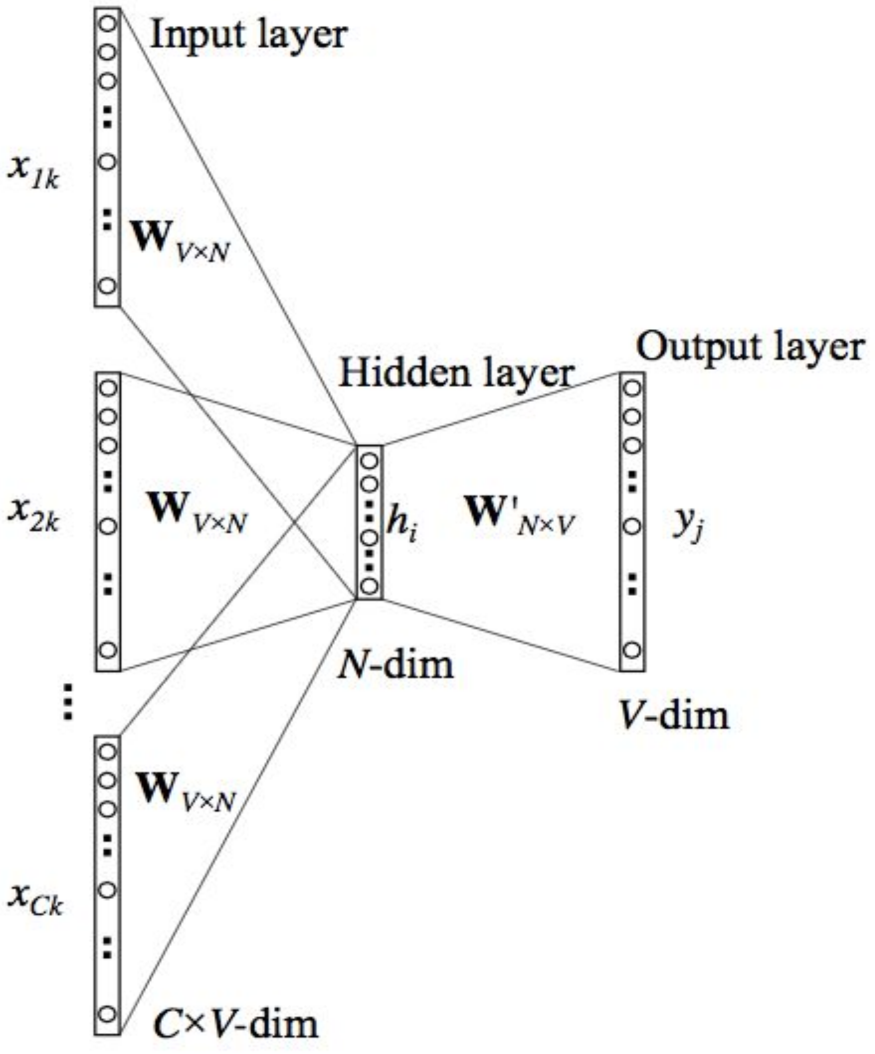


CBOW Architecture

The CBOW architecture is pretty simple contains :

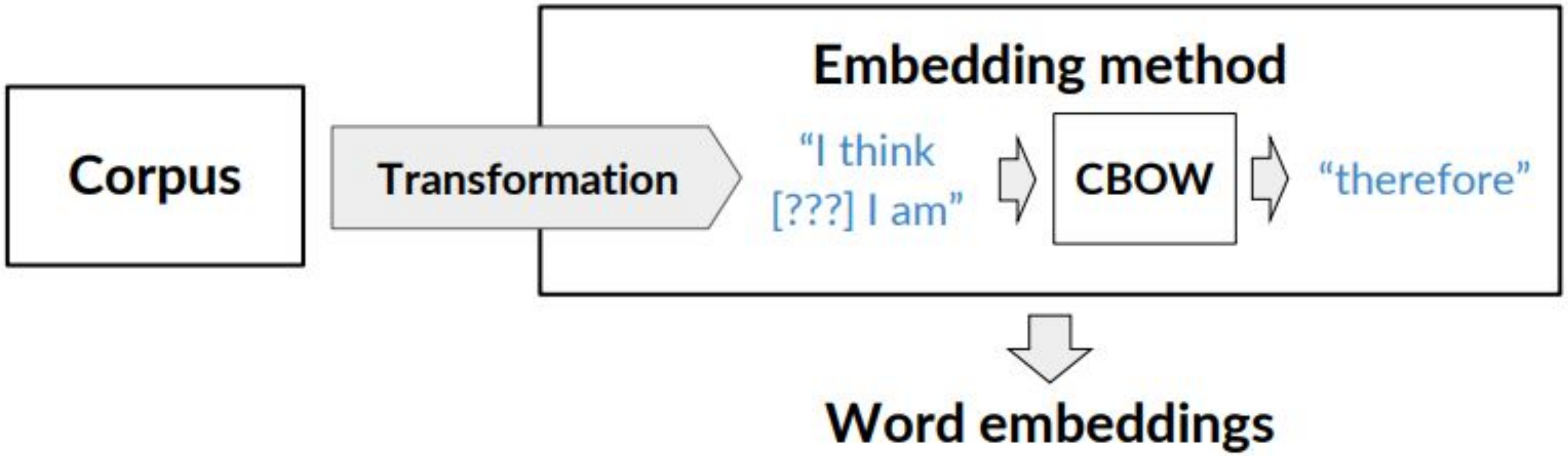
```
- the word embeddings as inputs (idx)
- the linear model as the hidden layer
- the softmax as the output
```





Token to features - CBOW

Continuous bag-of-words word embedding process



Token to features - CBOW

Corpus | Transformation | CBOW

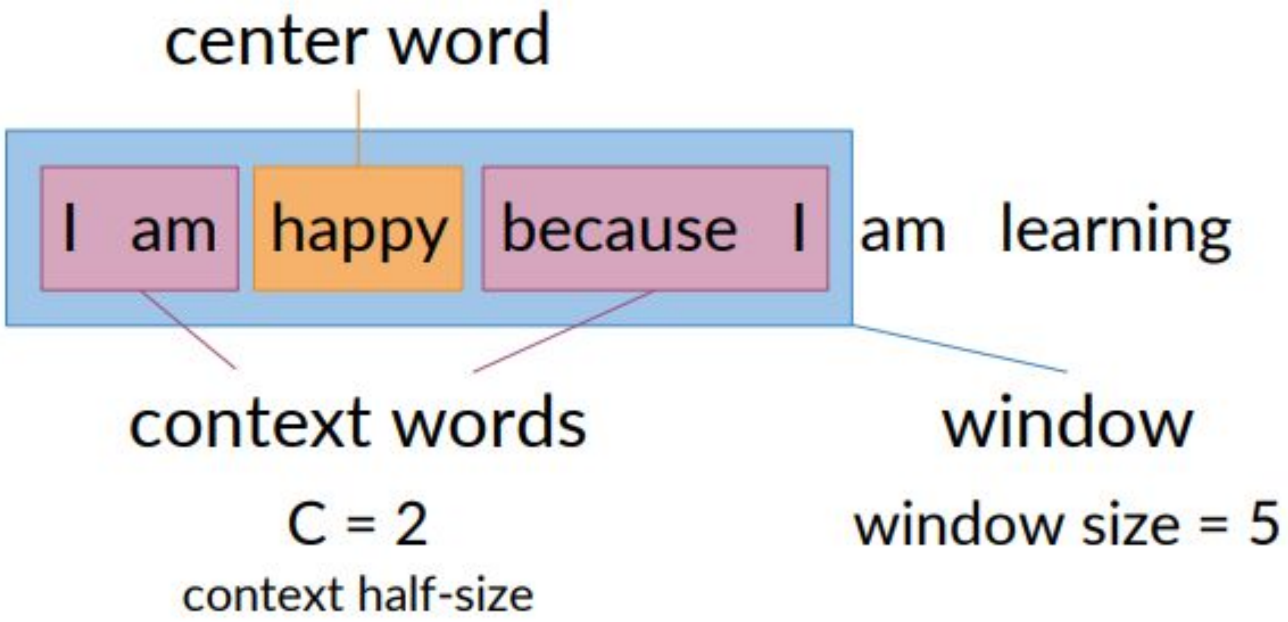
Center word prediction: rationale

The little ______?______ is barking

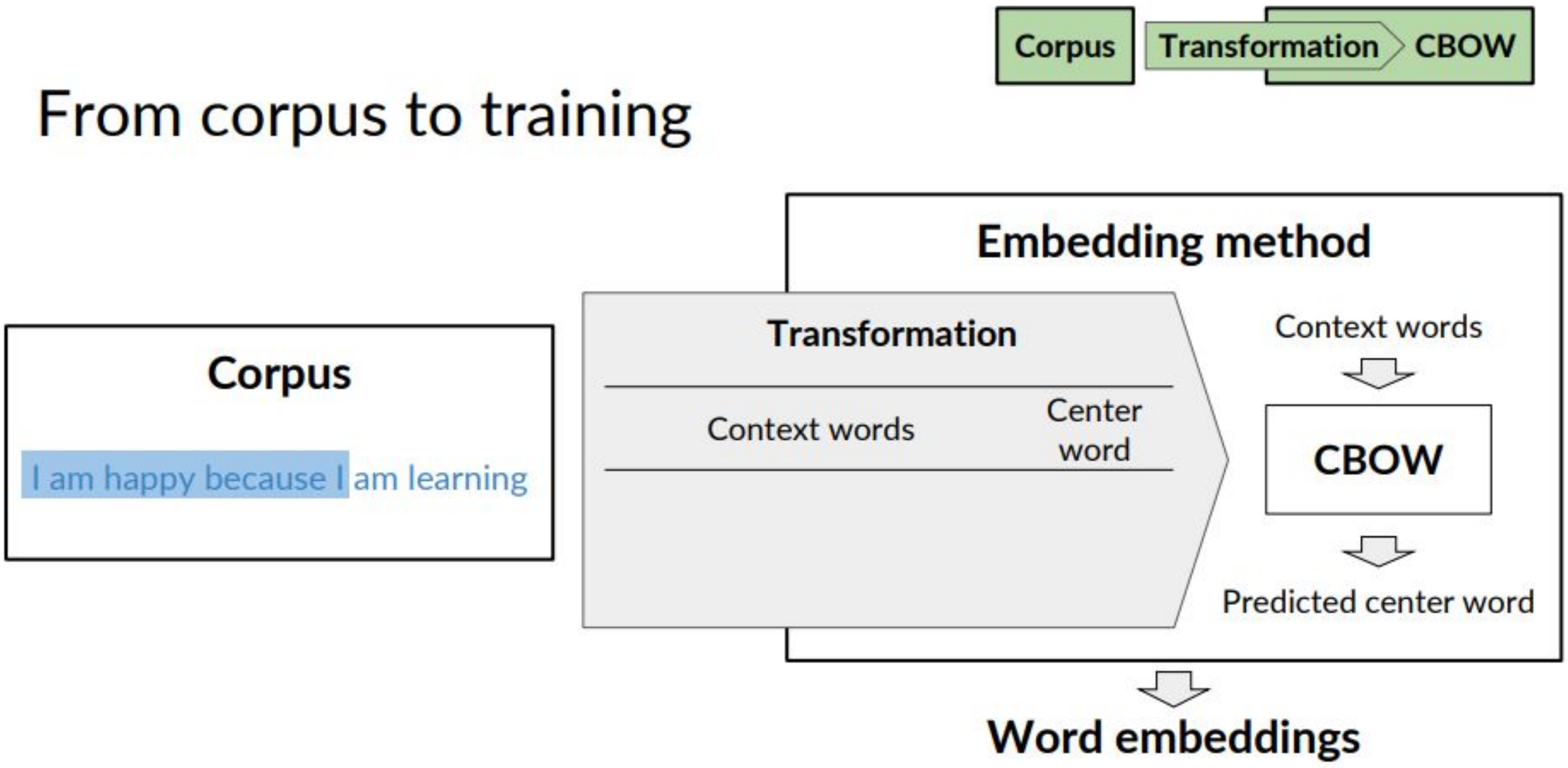
dog
puppy
hound
terrier
...

Token to features - CBOW

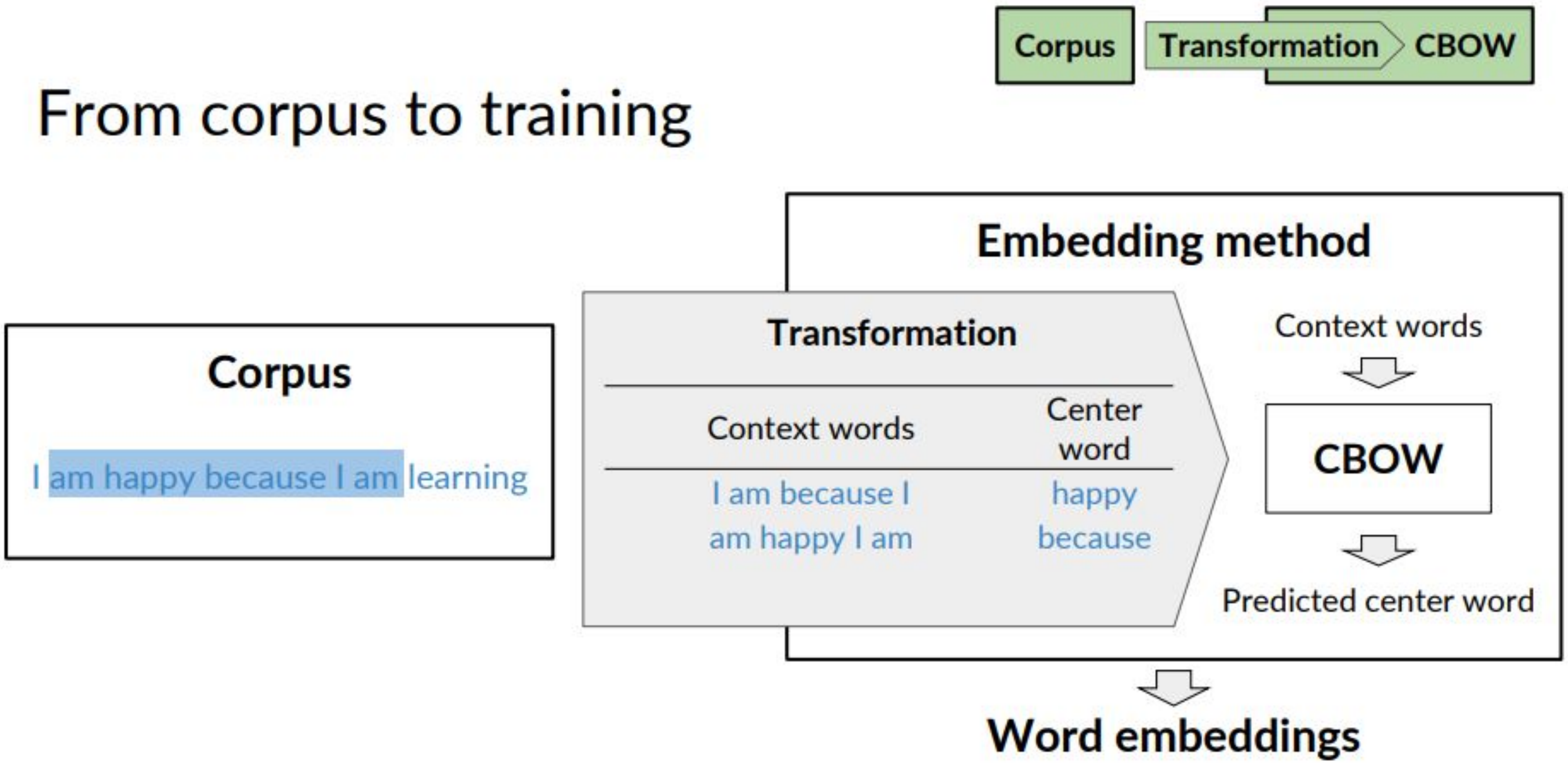
Creating a training example



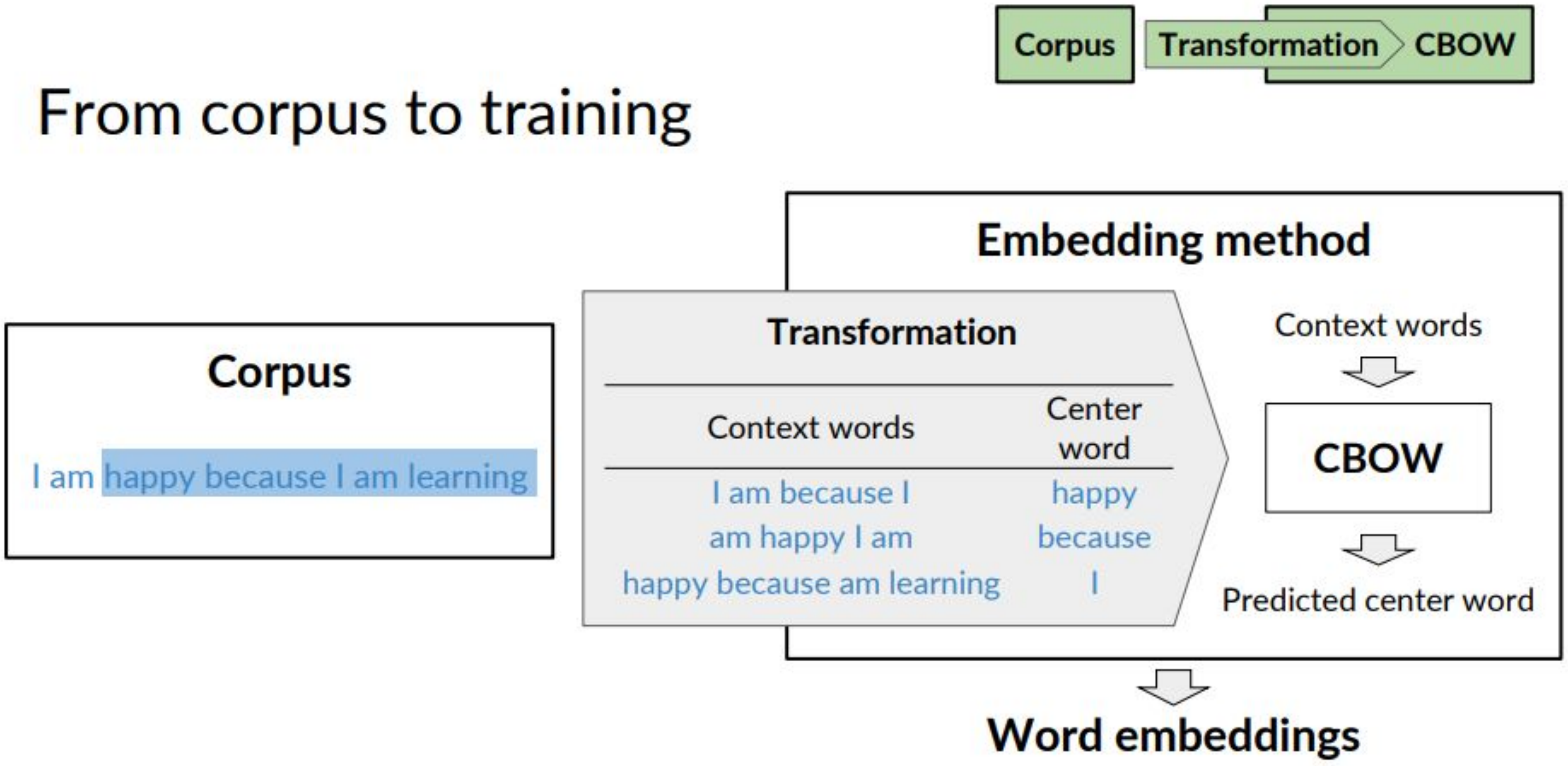
Token to features - CBOW



Token to features - CBOW

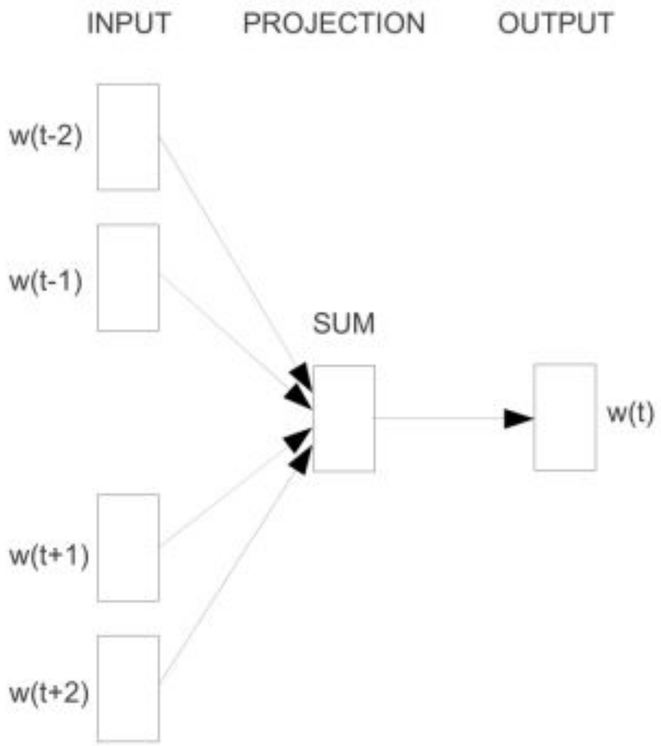


Token to features - CBOW



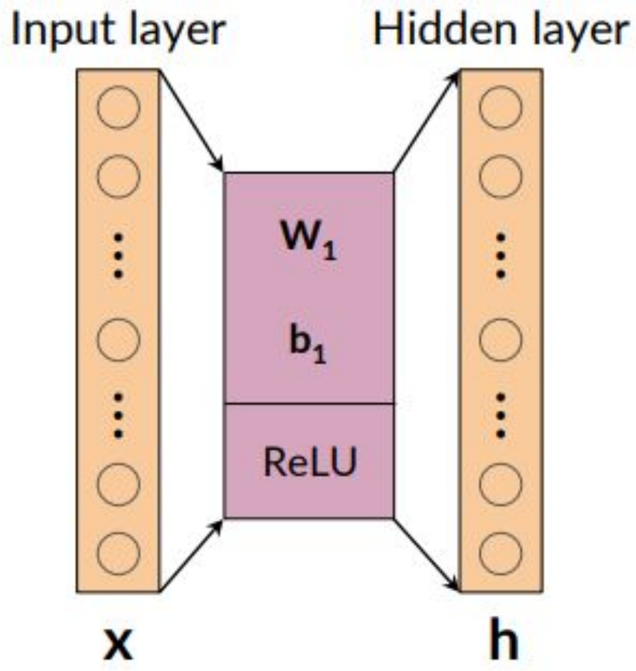
Token to features - CBOW

CBOW in a nutshell



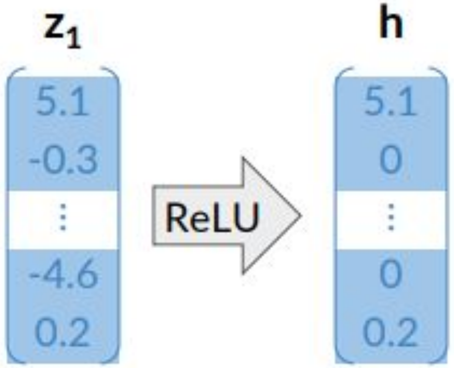
Source: Mikolov, T., Chen, K., Corrado, G.S., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space

Rectified Linear Unit (ReLU)

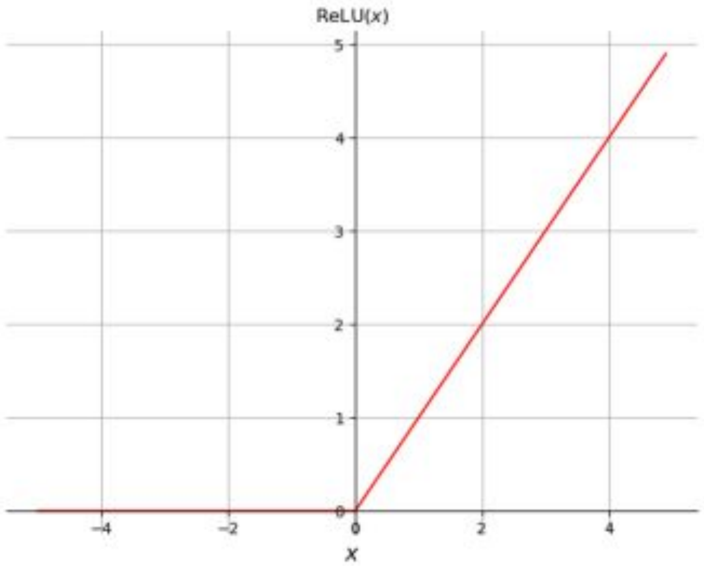


$$z_1 = W_1 x + b_1$$

$$h = \mathrm{ReLU}(z_1)$$

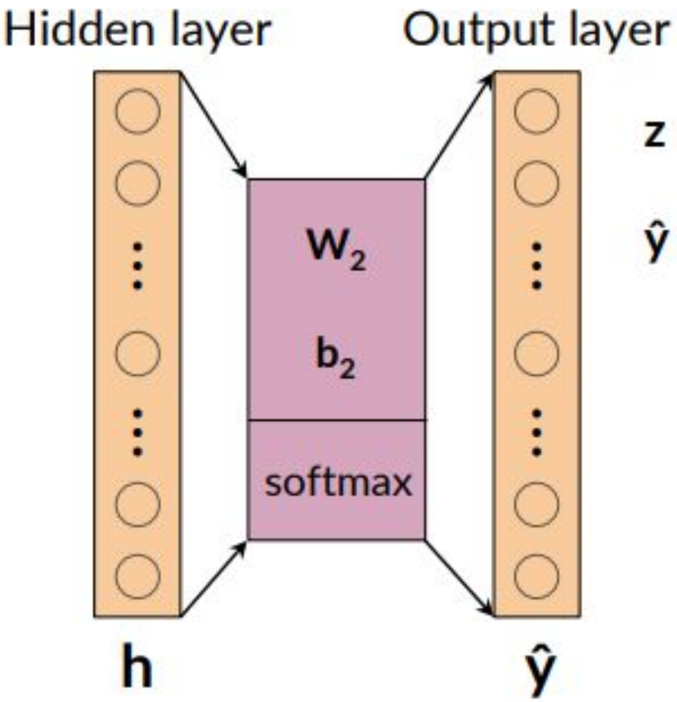


$$\mathrm{ReLU}(x) = \max(0, x)$$



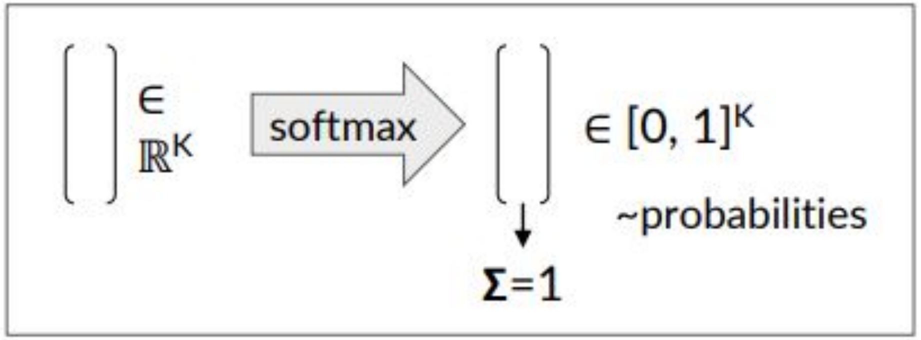
Token to features - CBOW

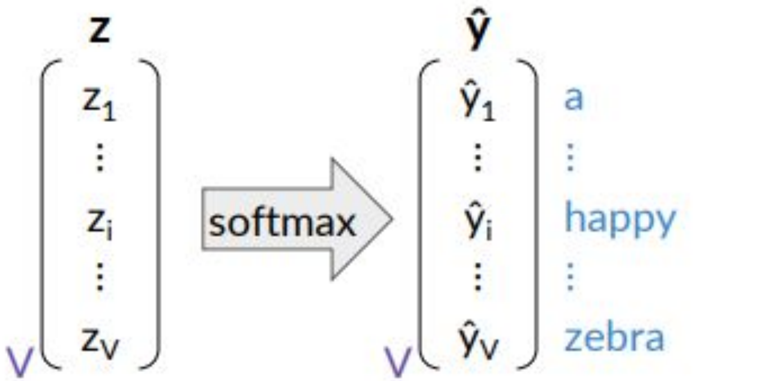
Softmax



$$z = W_2 h + b_2$$

$$\hat{y} = \text{softmax}(z)$$





Probabilities of being center word

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

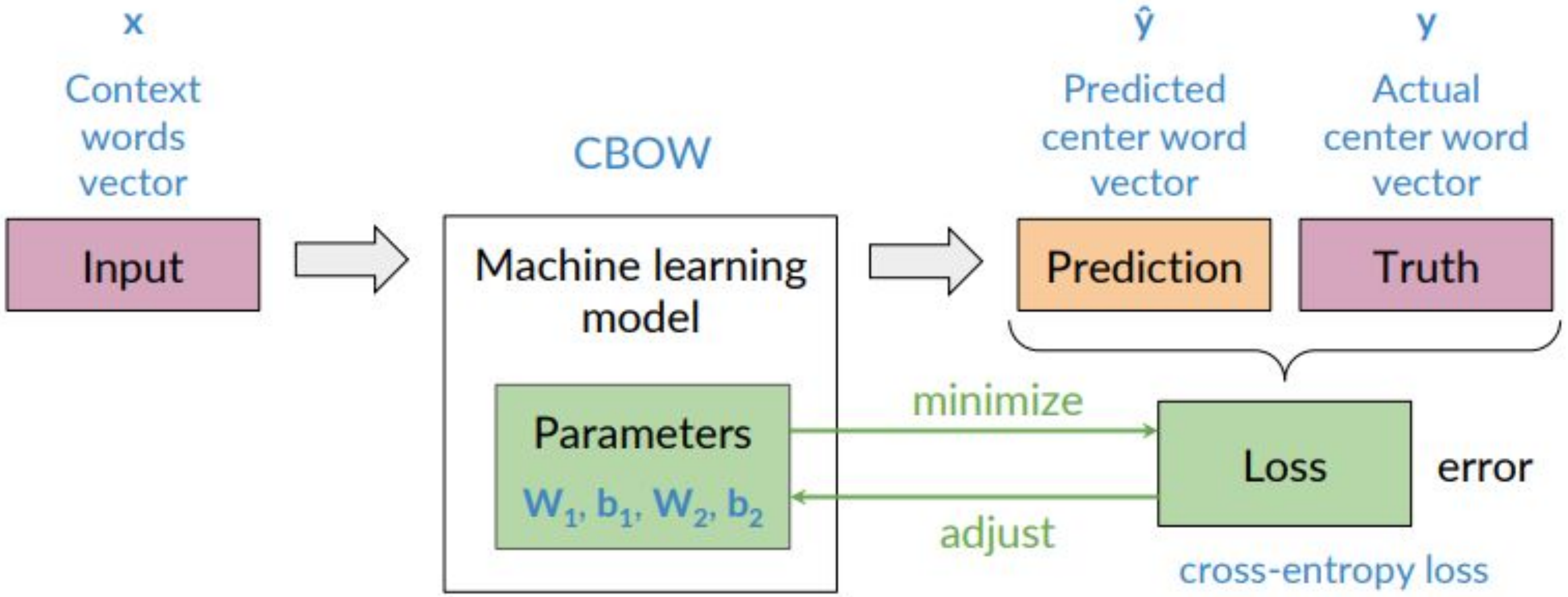
Token to features - CBOW

Softmax: example

$$\hat{y}_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

z		exp(z)		ŷ = softmax(z)	
9	exp	8103	/ Σ	0.083	am
8		2981		0.03	because
11		59874		0.612	happy ← Predicted center word
10		22026		0.225	I
8.5		4915		0.05	learning
		Σ=97899		Σ=1	

Loss


x
Context words vector
Input
CBOW
Machine learning model
Parameters
W1, b1, W2, b2
ŷ
Predicted center word vector
Prediction
y
Actual center word vector
Truth
minimize
adjust
Loss
error
cross-entropy loss

Token to features - CBOW

Cross-entropy loss

$$J = -\sum_{k=1}^{V} y_k \log \hat{y}_k$$

Actual $\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_V \end{pmatrix}$ Predicted $\hat{\mathbf{y}} = \begin{pmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_V \end{pmatrix}$

I am happy because I am learning

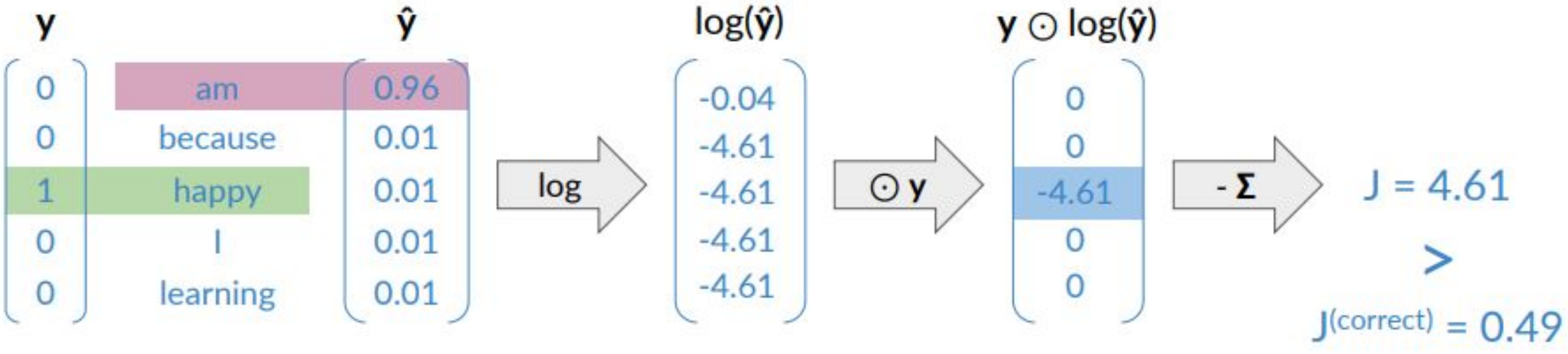
y		ŷ	log(ŷ)	y ⊙ log(ŷ)
0	am	0.083	-2.49	0
0	because	0.03	-3.49	0
1	happy	0.611	-0.49	-0.49
0	I	0.225	-1.49	0
0	learning	0.05	-2.49	0

$\hat{\mathbf{y}} \xrightarrow{\log} \log(\hat{\mathbf{y}}) \xrightarrow{\odot\, \mathbf{y}} \mathbf{y} \odot \log(\hat{\mathbf{y}}) \xrightarrow{-\Sigma} J = 0.49$

Token to features - CBOW

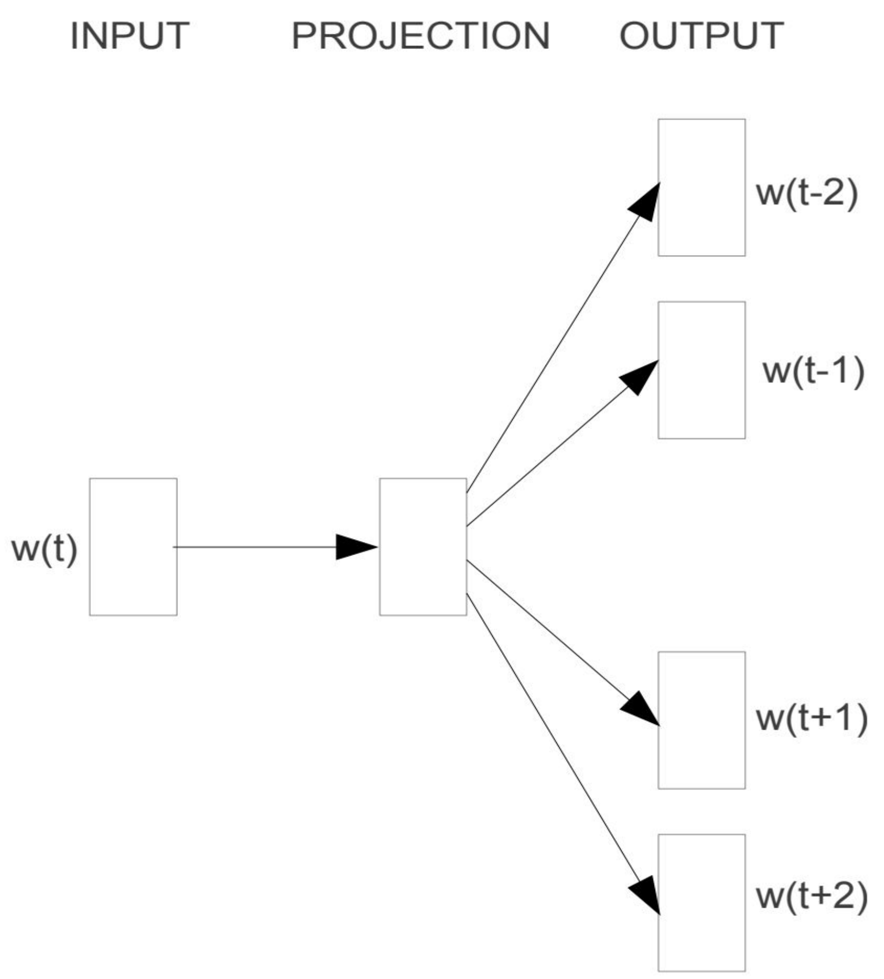
Cross-entropy loss

$$J = -\sum_{k=1}^{V} y_k \log \hat{y}_k$$



Skip-Gram:

- Predicts context words using a target word.
- Target = "coffee"; Context = "morning," "drink."
- Trains on (target, context) pairs across a window of neighboring words.



Skip-gram

Skip-Gram Example

Consider the following sentence of eight words:

The wide road shimmered in the hot sun.

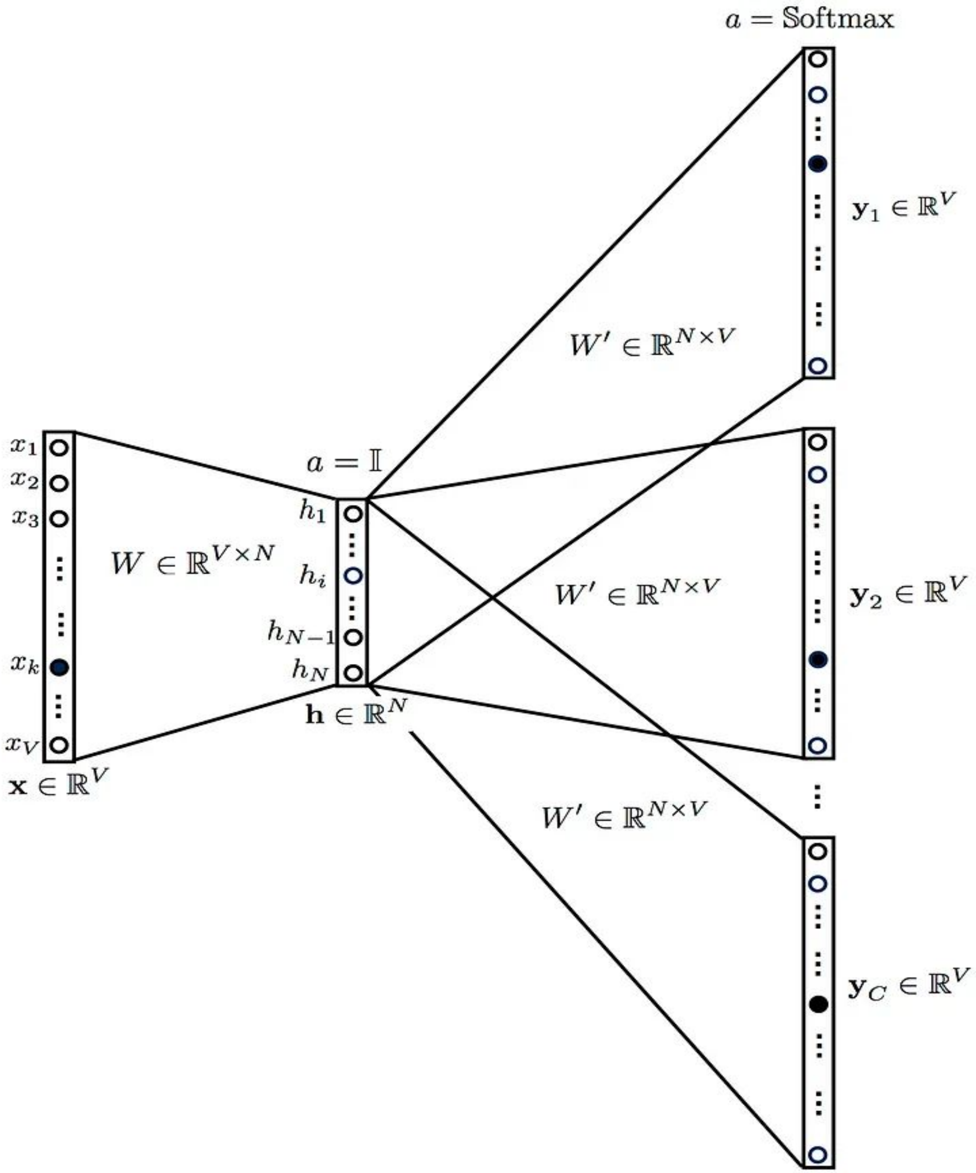
Window Size: Determines the number of words on each side of a target word that can be context words.

Skip-Grams: List of (target_word, context_word) pairs varies based on window size.

Token to features - SkipGram

Window Size	Text	Skip-grams
2	[The **wide** road shimmered] in the hot sun.	wide, the wide, road wide, shimmered
	The [wide road **shimmered** in the] hot sun.	shimmered, wide shimmered, road shimmered, in shimmered, the
	The wide road shimmered in [the hot **sun**].	sun, the sun, hot
3	[The **wide** road shimmered in] the hot sun.	wide, the wide, road wide, shimmered wide, in
	[The wide road **shimmered** in the hot] sun.	shimmered, the shimmered, wide shimmered, road shimmered, in shimmered, the shimmered, hot
	The wide road shimmered [in the hot **sun**].	sun, in sun, the sun, hot

Skip-Gram


$a = \text{Softmax}$
$\mathbf{y}_1 \in \mathbb{R}^V$
$W' \in \mathbb{R}^{N \times V}$
x_1
x_2
x_3
x_k
x_V
$\mathbf{x} \in \mathbb{R}^V$
$W \in \mathbb{R}^{V \times N}$
$a = \mathbb{I}$
h_1
h_i
h_{N-1}
h_N
$\mathbf{h} \in \mathbb{R}^N$
$W' \in \mathbb{R}^{N \times V}$
$\mathbf{y}_2 \in \mathbb{R}^V$
$W' \in \mathbb{R}^{N \times V}$
$\mathbf{y}_C \in \mathbb{R}^V$